

Usuários, Grupos e Permissões no Linux

Disciplina: Computação em Nuvem — Aula prática

Parte 1 — Teoria

1.1 Por que isso importa para Computação em Nuvem?

Antes de chegarmos a containers e VMs, precisamos entender **como o Linux controla quem pode acessar o quê**. Esse é exatamente o mesmo princípio por trás do **IAM** (Identity and Access Management) que vocês vão encontrar em qualquer provedor de nuvem (AWS, GCP, Azure): usuários, grupos, papéis (roles) e permissões mínimas necessárias — o chamado **princípio do menor privilégio**.

Se vocês entenderem bem `useradd`, `groupadd` e `chmod` hoje, vão reconhecer a mesma lógica quando virem "IAM Users", "IAM Groups" e "Policies" na AWS mais adiante.

1.2 Usuários no Linux

Todo usuário no Linux tem:

- Um **nome de login** (ex.: `joao`)
- Um **UID** (User ID) — número único que identifica o usuário. `0` é sempre o `root`.
- Um **grupo primário** (GID principal)
- Um **diretório home** (ex.: `/home/joao`)
- Um **shell padrão** (ex.: `/bin/bash`)

Essas informações ficam registradas no arquivo `/etc/passwd`. Cada linha tem este formato:

`joao:x:1001:1001:João Silva:/home/joao:/bin/bash`

Campo	Significado
<code>joao</code>	nome de usuário
<code>x</code>	senha (armazenada de fato, criptografada, em <code>/etc/shadow</code>)
<code>1001</code>	UID
<code>1001</code>	GID do grupo primário

Campo	Significado
João Silva	comentário/nome completo
/home/joao	diretório home
/bin/bash	shell padrão

Root (UID 0) é o superusuário — tem acesso irrestrito. No dia a dia, **não se loga como root diretamente**; usa-se **sudo** para executar comandos pontuais com privilégio elevado.

1.3 Grupos no Linux

Um **grupo** é uma forma de reunir vários usuários para dar a todos eles o mesmo tipo de acesso a arquivos e diretórios, sem precisar configurar permissão usuário por usuário.

- Cada usuário tem **um grupo primário** (definido na criação do usuário).
- Um usuário pode pertencer a **vários grupos secundários** também.

As informações de grupos ficam em **/etc/group**:

devs:x:1002:joao,maria,pedro

Campo	Significado
devs	nome do grupo
x	senha do grupo (raramente usada)
1002	GID
joao,maria,pedro	membros secundários do grupo

1.4 Permissões de arquivos e diretórios

Ao rodar `ls -l`, vocês veem algo assim:

```
-rwxr-xr-- 1 joao devs 4096 fev 20 10:00 script.sh
```

Vamos decompor:

```
-   rwx       r-x       r--  
|           |           |  
|           |           |
```

tipo dono grupo outros

- **Tipo:** - arquivo comum, `d` diretório, `l` link simbólico.
- **Três blocos de 3 caracteres**, cada um representando **read (r)**, **write (w)**, **execute (x)** para: **dono do arquivo, grupo do arquivo, outros usuários**.

Permissão	Em arquivo	Em diretório
<code>r</code> (read)	ler o conteúdo	listar o conteúdo (<code>ls</code>)
<code>w</code> (write)	modificar/apagar	criar/remover arquivos dentro
<code>x</code> (execute)	executar como programa	"entrar" no diretório (<code>cd</code>)

1.5 Notação numérica (octal)

Cada permissão vale um número: $r = 4$, $w = 2$, $x = 1$. Soma-se para cada classe:

Combinação	Valor
rwX	$4+2+1 = 7$
$rw-$	$4+2+0 = 6$
$r-x$	$4+0+1 = 5$
$r--$	$4+0+0 = 4$
$---$	0

Exemplo: $rwXr-xr-- = 754$ (dono=7, grupo=5, outros=4).

1.6. Exercícios: Usuários e Grupos

Execute os comandos abaixo como **root** ou utilizando sudo:

1. Criar grupos e usuários

Cria dois grupos

```
sudo groupadd devs
```

```
sudo groupadd sysadmin
```

Cria dois usuários com diretório home (-m)

```
sudo useradd -m carlos
```

```
sudo useradd -m ana
```

Define uma senha inicial para cada um

```
sudo passwd carlos
```

```
sudo passwd ana
```

2. Adicionar usuários aos grupos

Adiciona o usuário carlos ao grupo 'devs' como grupo secundário (-aG)

```
sudo usermod -aG devs carlos
```

Adiciona a usuária ana aos grupos 'devs' e 'sysadmin'

```
sudo usermod -aG devs,sysadmin ana
```

3. Inspeccionar as alterações nos arquivos de sistema Veja como o sistema registrou os novos usuários e grupos:

Verifique as últimas linhas de cada arquivo

```
tail -n 5 /etc/passwd
```

```
tail -n 5 /etc/group
```

- **No /etc/passwd:** Você verá linhas no formato carlos:x:1001:1001::/home/carlos:/bin/bash, contendo: nome de usuário, flag de senha (x), UID, GID principal, diretório home e shell padrão.
- **No /etc/group:** Você verá linhas no formato devs:x:1002:carlos,ana, listando o nome do grupo, GID e os usuários adicionados como membros secundários separados por vírgula.

4. Remover usuários dos grupos

Remove o usuário carlos apenas do grupo 'devs'

```
sudo gpasswd -d carlos devs
```

Inspeccione novamente com grep devs /etc/group para confirmar que carlos sumiu da lista de membros do grupo.

5. Deletar usuários e grupos

Remove o usuário carlos e seu diretório home (-r)

```
sudo userdel -r carlos
```

Remove a usuária ana e seu diretório home (-r)

```
sudo userdel -r ana
```

```
# Remove os grupos que não são mais necessários
```

```
sudo groupdel devs
```

```
sudo groupdel sysadmin
```

6. Verificar o estado final

```
# Confirme que os registros foram completamente removidos
```

```
tail -n 5 /etc/passwd
```

```
tail -n 5 /etc/group
```

- As entradas de carlos e ana foram apagadas de /etc/passwd.
- Os grupos devs e sysadmin (junto com suas referências) foram removidos de /etc/group.

1.7. Exercícios: Permissões no Linux

As permissões no Linux controlam leitura (**r** = 4), escrita (**w** = 2) e execução (**x** = 1) para o dono (**u**), o grupo (**g**) e outros (**o**).

1. Preparar o ambiente

```
# Cria um arquivo de teste e um grupo
touch relatorio.txt
sudo groupadd financeiro
```

```
# Lista os detalhes para ver o dono, grupo e permissões padrão
ls -l relatorio.txt
A saída será algo como: -rw-r--r-- 1 usuario usuario 0 relatorio.txt
```

2. Alterar o dono e grupo com chown

O comando `chown` (*change owner*) altera o usuário dono e, opcionalmente, o grupo.

```
# Muda apenas o dono do arquivo para 'carlos' (ou outro usuário existente)
sudo chown root relatorio.txt
```

```
# Muda o dono e o grupo ao mesmo tempo (sintaxe: usuario:grupo)
sudo chown root:financeiro relatorio.txt
```

```
# Confirme a alteração
ls -l relatorio.txt
```

3. Alterar apenas o grupo com chgrp

O comando `chgrp` (*change group*) serve exclusivamente para transferir o arquivo para outro grupo sem mexer no dono.

```
# Altera o grupo do arquivo
sudo chgrp financeiro relatorio.txt
```

```
# Confirme a alteração
ls -l relatorio.txt
```

4. Modificar permissões com chmod (Modo Simbólico)

Usa letras para conceder (+), revogar (-) ou definir (=):

```
# Dá permissão de escrita (w) para o grupo (g)
sudo chmod g+w relatorio.txt
```

```
# Remove a permissão de leitura (r) para outros/resto do sistema (o)
sudo chmod o-r relatorio.txt
```

```
# Torna o arquivo executável apenas para o dono (u)
sudo chmod u+x relatorio.txt
```

Inspecione o resultado
ls -l relatorio.txt

5. Modificar permissões com chmod (Modo Octal / Numérico)

Soma-se os valores desejados para cada nível (Dono | Grupo | Outros):

- 4 = Leitura (r)
- 2 = Escrita (w)

7 (4+2+1 = rwx) para o dono | 5 (4+0+1 = r-x) para o grupo | 0 (nenhuma) para outros
sudo chmod 750 relatorio.txt

6 (4+2 = rw-) para o dono | 4 (leitura) para grupo | 4 (leitura) para outros
sudo chmod 644 relatorio.txt

Confirme as novas permissões
ls -l relatorio.txt

Tabela de Permissões Octais Comuns

Código	Permissão Simbólica	Descrição
777	rw-rw-rwx	Acesso total para todos (leitura, escrita e execução).
755	rw-r-xr-x	Dono controla tudo; grupo e outros apenas leem e executam.
644	rw-r--r--	Padrão de arquivos de texto: dono edita, demais apenas leem.
600	rw-----	Privado: apenas o dono lê e edita (comum para chaves SSH).

1.8 Comandos essenciais

Comando	Função
<code>sudo useradd -m -G grupo usuario</code>	cria usuário com home (-m) e adiciona a um grupo secundário (-G)
<code>sudo passwd usuario</code>	define/altera senha do usuário
<code>sudo usermod -aG grupo usuario</code>	adiciona usuário a um grupo sem removê-lo dos outros (-a = append)
<code>sudo groupadd grupo</code>	cria um grupo novo
<code>groups usuario</code>	mostra a quais grupos o usuário pertence
<code>id usuario</code>	mostra UID, GID e todos os grupos
<code>su - usuario</code>	troca para outro usuário (sessão de login completa)
<code>chmod</code>	altera permissões (simbólico <code>u+x</code> ou numérico <code>755</code>)
<code>chown usuario:grupo arquivo</code>	altera dono e grupo de um arquivo
<code>chgrp grupo arquivo</code>	altera apenas o grupo
<code>umask</code>	define as permissões padrão de arquivos/diretórios novos

Todo comando que cria/altera usuários, grupos ou dono de arquivo exige **sudo**.

Parte 2 — Exercício prático

Cenário

Vocês foram contratados como administradores de sistemas da empresa fictícia **TechNuvem Ltda.**, que está estruturando o acesso da sua equipe de TI a um novo servidor Linux. Vocês vão criar toda a estrutura de usuários, grupos e permissões do zero.

A equipe da empresa é composta por três times:

- **ti-admin** — administradores de infraestrutura (acesso total aos projetos)
- **devs** — desenvolvedores (podem ler e escrever nos projetos, mas não administrar)
- **auditoria** — time de compliance (pode **apenas ler** os projetos, nunca alterar)

Façam cada etapa **na ordem**, e **anotem os comandos usados e os resultados** — vocês vão precisar disso para responder as perguntas finais.

Etapa 1 — Reconhecimento do sistema atual

1. Descubra qual usuário vocês estão usando agora: **whoami**
2. Veja o UID, GID e grupos do usuário atual: **id**
3. Liste todos os usuários existentes no sistema: **cat /etc/passwd**
4. Liste todos os grupos existentes: **cat /etc/group**
5. Identifiquem, olhando o arquivo **/etc/passwd**, qual é o UID do **root** e por que ele é especial.

Etapa 2 — Criando os grupos da empresa

Criem os três grupos do cenário:

```
sudo groupadd ti-admin
sudo groupadd devs
sudo groupadd auditoria
```

Confirmem que os três grupos foram criados:

```
cat /etc/group | grep -E "ti-admin|devs|auditoria"
```

Etapa 3 — Criando os usuários

Criem **6 usuários**, dois para cada time, cada um já associado ao grupo secundário correto:

```
sudo useradd -m -G ti-admin -s /bin/bash carla
sudo useradd -m -G ti-admin -s /bin/bash bruno
sudo useradd -m -G devs -s /bin/bash joao
sudo useradd -m -G devs -s /bin/bash maria
sudo useradd -m -G auditoria -s /bin/bash pedro
sudo useradd -m -G auditoria -s /bin/bash ana
```

Definam uma senha para **cada um** dos 6 usuários (o sistema vai pedir para digitar duas vezes):

```
sudo passwd carla
sudo passwd bruno
sudo passwd joao
sudo passwd maria
sudo passwd pedro
sudo passwd ana
```

Verificação: rodem `id joao` e `id pedro`. Confirmem que cada um aparece com o grupo secundário correto.

Etapa 4 — Explorando o `su` (trocar de usuário)

1. Troquem para o usuário `joao`:

```
su - joao
```

2. Rodem `whoami` e `pwd` — reparem que agora vocês estão na home do `joao` (`/home/joao`).
 3. Tentem rodar `sudo apt update`. **Deve dar erro de permissão** — por quê isso acontece? (Dica: `joao` não está no grupo `sudo`.)
 4. Voltem para o usuário original digitando `exit`.
-

Etapa 5 — Criando a estrutura de diretórios compartilhados

Ainda como o usuário original (com `sudo`), criem a estrutura de projetos da empresa:

```
sudo mkdir -p /projetos/webapp
sudo mkdir -p /projetos/relatorios-auditoria
```

Etapa 6 — Ajustando dono e grupo dos diretórios

O diretório `/projetos/webapp` deve pertencer ao grupo `devs`:

```
sudo chown root:devs /projetos/webapp
```

O diretório `/projetos/relatorios-auditoria` deve pertencer ao grupo `auditoria`:

```
sudo chown root:auditoria /projetos/relatorios-auditoria
```

Verificação: rodem `ls -ld /projetos/webapp /projetos/relatorios-auditoria` e confirmem o dono/grupo de cada um.

Etapa 7 — Configurando permissões (o coração do exercício)

Agora vamos definir exatamente quem pode fazer o quê. Apliquem as seguintes regras usando `chmod`:

Para `/projetos/webapp`:

- Dono (`root`): leitura, escrita e execução → `rwX`
- Grupo (`devs`): leitura, escrita e execução → `rwX`
- Outros: nenhuma permissão → `---`

```
sudo chmod 770 /projetos/webapp
```

Para `/projetos/relatorios-auditoria`:

- Dono (`root`): leitura, escrita e execução → `rwX`
- Grupo (`auditoria`): **apenas** leitura e execução (podem entrar e ver arquivos, mas não criar/apagar) → `r-X`
- Outros: nenhuma permissão → `---`

```
sudo chmod 750 /projetos/relatorios-auditoria
```

Verificação: `ls -ld /projetos/webapp /projetos/relatorios-auditoria` deve mostrar `drwxrwx---` e `drwxr-x---` respectivamente.

Etapa 8 — Testando as permissões na prática

Agora o teste de verdade. Façam cada um destes testes e **anotem se funcionou ou deu erro**:

#	Ação	Comando	Resultado esperado
1	joao (devs) cria um arquivo em /projetos/webapp	su - joao → touch /projetos/webapp/app.py	Deve funcionar
2	pedro (auditoria) tenta criar um arquivo em /projetos/webapp	su - pedro → touch /projetos/webapp/teste.txt	Deve falhar (Permission denied)
3	pedro (auditoria) tenta ler /projetos/relatorios-auditoria	su - pedro → ls /projetos/relatorios-auditoria	Deve funcionar
4	pedro (auditoria) tenta criar um arquivo em /projetos/relatorios-auditoria	su - pedro → touch /projetos/relatorios-auditoria/relatorio.txt	Deve falhar
5	joao (devs) tenta acessar /projetos/relatorios-auditoria	su - joao → ls /projetos/relatorios-auditoria	Deve falhar

Depois de cada `su - usuario`, não esqueçam de digitar `exit` para voltar.

Etapa 9 — O bit SETGID (herança de grupo)

Um problema comum: quando `maria` cria um arquivo novo em `/projetos/webapp`, o **grupo do arquivo criado é o grupo primário dela**, não `devs` — o que pode quebrar as permissões para os outros membros do time.

A solução é o **bit SETGID** no diretório, que faz todo arquivo novo criado dentro dele herdar automaticamente o grupo do diretório:

```
sudo chmod g+s /projetos/webapp
```

Verificação:

1. Rodem `ls -ld /projetos/webapp` — reparem que agora aparece `drwxrws---` (o `s` no lugar do `x` do grupo).

2. Entrem como `maria` (`su - maria`) e criem um arquivo: `touch /projetos/webapp/estilo.css`
 3. Saiam (`exit`) e rodem `ls -l /projetos/webapp` — confirmem que o arquivo `estilo.css` pertence ao grupo `devs`, mesmo `maria` não tendo especificado isso.
-

Etapa 10 — Umask: permissões padrão

O `umask` define quais permissões são **subtraídas** por padrão de todo arquivo/diretório novo criado por um usuário.

1. Como usuário comum, rodem: `umask` — anotem o valor mostrado (geralmente `0022`).
2. Criem um arquivo de teste: `touch teste_umask.txt` e rodem `ls -l teste_umask.txt`. Por padrão, novo arquivo nunca nasce com `x`, então o resultado costuma ser `rw-r--r--` (644).
3. Agora mudem o umask temporariamente: `umask 077`
4. Criem outro arquivo: `touch teste_umask2.txt` e comparem: `ls -l teste_umask2.txt` deve mostrar `rw-----` (600) — só o dono tem acesso.

Pergunta: por que é uma boa prática de segurança usar `umask 077` em servidores que lidam com dados sensíveis?

Desafio final (sem passo a passo)

Sozinhos ou em dupla, sem consultar as etapas anteriores:

1. Criem um novo grupo chamado **estagiarios**.
 2. Criem um usuário **lucas**, associado a esse grupo.
 3. Criem um diretório **/projetos/sandbox** que:
 - pertença ao grupo **estagiarios**;
 - permita ao grupo **ler e escrever, mas não executar/entrar** — pensem em qual permissão numérica representa isso;
 - **não dê nenhum acesso a outros usuários**.
 4. Testem como **lucas** se ele consegue criar um arquivo dentro do **/projetos/sandbox**. O que acontece e por quê? (Dica: pensem no que a permissão **x** significa em diretórios.)
-

Limpeza (opcional, ao final da aula)

Se estiverem em uma VM de testes, podem remover tudo o que criaram:

```
sudo userdel -r carla
sudo userdel -r bruno
sudo userdel -r joao
sudo userdel -r maria
sudo userdel -r pedro
sudo userdel -r ana
sudo userdel -r lucas
sudo groupdel ti-admin
sudo groupdel devs
sudo groupdel auditoria
sudo groupdel estagiarios
sudo rm -rf /projetos
```

`userdel -r` remove o usuário e seu diretório home.

Perguntas de reflexão (entregar por escrito)

1. Qual a diferença entre grupo **primário** e grupo **secundário** de um usuário?
 2. Por que a permissão `x` em um diretório é necessária mesmo que o usuário já tenha `r`? O que acontece se um diretório tem `r` mas não `x`?
 3. Expliquem, com suas palavras, o que o bit **SETGID** resolveu na Etapa 9 e por que isso é importante em ambientes de equipe.
 4. Relacionem o que fizeram hoje (grupos `ti-admin`, `devs`, `auditoria`) com o conceito de **IAM Groups** e **princípio do menor privilégio** em provedores de nuvem. Por que a nuvem usa essa mesma lógica?
 5. Na Etapa 8, o teste #2 falhou (pedro não conseguiu criar arquivo em `/projetos/webapp`). Qual comando de verificação (`ls -ld`) explica exatamente por que isso aconteceu?
-

Próximo tema: instalação de pacotes com `apt` — e aí, sim, vamos preparar o terreno para o Docker.